Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Quang-Vinh Dang[a,*]

[a] *School of Innovative and Computing Sciences, British University Vietnam, Hung Yen, Vietnam*

Abstract

Gradient-boosted decision trees dominate flow-based network intrusion detection, yet their robustness to physically realizable evasion is rarely measured at scale. Feature-space attacks ignore the coupling invariants of flow records and produce telemetry that no packet sequence can generate, while stochastic packet mutators need hundreds of queries. We propose Flow-Field Evasion (FFE), which models four protocol-compliant actions (temporal dilation, payload padding, chaff injection, TCP window morphing) as infinitesimal generators of a Lie cone on the flow manifold, steers along the cheapest generator, and snaps to tree split thresholds. On the complete, deduplicated CSE-CIC-IDS2018 (4.3 million flows), CIC-IDS2017 (1.7 million) and UNSW-NB15 (120 thousand) captures, FFE evades XGBoost in 100.0%, 95.9% and 61.5% of detected attacks, all physically feasible, with 13–91 queries. It outperforms Traffic Manipulator PSO and CAA/MOEVA on every corpus; against surrogate-based CAPGD it is significantly better on

*Corresponding author

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

two corpora and statistically tied on CSE-CIC-IDS2018 (100.0% vs. 99.5%), where CAPGD needs fewer queries. Lie-guided adversarial training lowers FFE success from 96.3% to 33.0% without clean-accuracy loss, whereas malleability penalties alone reach 67.0%.

Keywords: Adversarial machine learning, Network intrusion detection, Gradient boosted decision trees, Evasion attacks, Problem-space constraints, Lie algebra, Adversarial training

1. Introduction

Modern network intrusion detection systems (NIDS) increasingly rely on machine learning over flow-based telemetry. At multi-gigabit rates, inspecting every packet payload is prohibitive, so sensors such as CICFlowMeter [1] aggregate bidirectional packet exchanges into tabular flow records of timing, length, header and flag statistics. Gradient-boosted decision trees (GBDTs), notably XGBoost [2], LightGBM [3] and CatBoost [4], are the dominant classifiers for these records.

Near-perfect accuracy on benchmark corpora does not guarantee robustness in operation. Closed-world evaluations reward models that exploit testbed artifacts [5] and shortcut features [6]. Attackers, however, control the malleable parts of their traffic, such as timing, padding and window sizes, so a detector that relies on these features can be evaded without losing the exploit [7].

Evading a tree ensemble is hard for two reasons. First, the ensemble is piecewise constant, so $\nabla_x F(x) = 0$ for almost all x and gradient-based attacks receive no signal. Second, flow features obey conservation laws. Flow

Bytes/s satisfies $x_8 = 10^6(x_4 + x_5)/x_6$, where x_4 and x_5 are the forward and backward byte totals and x_6 is the duration in microseconds. A perturbation that changes the byte totals without updating the rate produces a phantom flow that protocol sanity checks reject [8, 7].

Existing work follows one of two paradigms, and neither is adequate. Feature-space methods such as ZOO [9] and HopSkipJump [10] ignore the physical coupling: evaluated against flow invariants on the complete CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures, HopSkipJump yields no physically feasible flow at all (0.0% Real ASR despite 100.0% nominal evasion on two corpora), and ZOO keeps at most 14.9% Real ASR. Constraint-aware search such as CAA/MOEVA [11, 12] and packet-level swarm search such as Traffic Manipulator [13] respect more of the physics, but query the classifier as a black box, needing 161–434 queries per flow. Recent surrogate-based attacks [11, 14] are cheap at attack time but depend on how closely a surrogate matches the target.

This article proposes Flow-Field Evasion (FFE), which combines both properties. FFE derives physical perturbations as differential generators, so the continuous flow satisfies the invariants exactly and a projection enforces them on the discrete schema, and it steers flows using the split geometry of the ensemble. To our knowledge, no prior attack on tree ensembles combines these two properties.

The main contributions are as follows.

- We model four packet-level attacker actions (temporal dilation, payload padding, chaff injection and TCP window morphing) as closed-form infinitesimal generators of a Lie cone on the flow manifold.

- We prove that trajectories in this cone preserve the rate, mean-length, header and MTU invariants of flow records in the continuous relaxation of the flow manifold.

- We present FFE, which moves each flow along the generator with the largest cost-weighted score decrease and finishes with tree-split boundary snapping (TSBS).

- We evaluate FFE at full scale on the complete, deduplicated CSE-CIC-IDS2018 (4.3 million flows), CIC-IDS2017 (1.7 million) and UNSW-NB15 (120 thousand) captures against XGBoost and LightGBM, with seven baselines including re-implementations of CAPGD and a NIDSGAN-style generator. FFE reaches a Real ASR of 100.0%, 95.9% and 61.5% against XGBoost with 100.0% physical feasibility and 13.0–91.4 queries per flow, significantly above PSO and CAA/MOEVA everywhere. Against CAPGD it is significantly better on two corpora and statistically tied on CSE-CIC-IDS2018 (100.0% vs. 99.5%, $p = 0.0625$), where CAPGD uses fewer queries. Evasion depends on the target: LightGBM on CIC-IDS2017 is evaded in 34.6% of the cohort against 95.9% for XGBoost.

- Lie-guided adversarial training reduces FFE success from 96.3% to 33.0% without clean-accuracy loss; masking malleable features alone leaves 67.0% and adds nothing on top of adversarial training.

2. Related Work and Problem Formulation

2.1. Related Work and State-of-the-Art Taxonomy

Gradient-boosted decision trees (GBDTs), including XGBoost [2], LightGBM [3] and CatBoost [4], are the standard choice for flow-based network intrusion detection on benchmarks such as CSE-CIC-IDS2018 [1], UNSW-NB15 [15] and CIDDS-001 [16]. Comprehensive empirical studies support this choice [17, 18, 7].

Early adversarial attacks on NIDS were transplanted from image classification. Szegedy et al. [19], Goodfellow et al. [20] and Papernot et al. [21] showed that gradient-based perturbations fool deep networks, and optimizers such as the Carlini-Wagner L_2 attack [22] were then applied to tabular flow features. These studies reported high success rates, but the perturbed vectors were not network flows: they assign fractional values to packet counts, modify immutable fields such as destination port, and break the conservation between byte totals and packet counts. The problem-space view corrects this. Pierazzi et al. [8] showed that an adversarial perturbation is meaningful only when it is realized as a transformation of the physical object, such as a binary or a packet sequence, and that the transformation must preserve functionality and survive the feature extractor. Verkerken et al. [23] apply this view to defining host-space adversarial perturbations against NIDS. Surveys and audits of this literature [7, 24, 25, 26] argue that many feature-space attacks assume threat models that cannot be realized with valid packets under RFC protocol specifications, so their success rates overstate the risk once evaluated against true packet parsers and middleboxes. Constrained tabular attacks address feasibility only in part. CAA [11] combines a gradient-based

attack (CAPGD) with MOEVA [12], a multi-objective genetic search whose constraint penalties are heuristic and whose population search receives no directional guidance. TabularBench [27] standardizes such evaluations across deep and tree models. Packet-level tools avoid these problems by operating on real traffic. Traffic Manipulator [13] uses particle swarm optimization to mutate PCAP files by perturbing inter-packet intervals, adding payload bytes and injecting chaff packets. IDSGAN [28] generates adversarial flow records with generative adversarial networks, but its feasibility is only approximated in feature space.

Recent work makes the realism requirement concrete for NIDS. Merzouk et al. [29] show that many published evasion attacks cannot be realized on live traffic, Sheatsley et al. [30] and Debicha et al. [31, 32] craft domain-constrained or botnet flows that remain valid, Zhang et al. [33] build explainable and transferable attacks, and El Shehaby et al. [34] score which flow features an attacker can perturb, the mutable/immutable split that FFE builds into its schema. Learned generators [28, 14] and grey-box frameworks [35] trade query cost for a training phase. For tabular data in general, CAPGD and CAA [11], cost-aware attacks [36] and recent benchmarks and surveys [27, 37, 38, 39] show that constraint handling dominates reported success rates. The tree-ensemble literature moved toward faster and deployable attacks [40, 41], evasion detection [42], and robust training [43, 44, 45, 46, 47, 48]. None of these exploits split geometry while keeping coupled flow invariants exact.

Evaluation practice has also been questioned: CIC-IDS2017 and CSE-CIC-IDS2018 contain labeling errors and duplicated records [49, 50, 51],

datasets differ widely in difficulty [52, 53], and detectors generalize poorly across datasets [54, 55]. We therefore evaluate on complete, deduplicated captures (Section 4.1).

Exact methods exploit the piecewise-constant structure of trees. Kantchelian et al. [56] cast minimal evasion against tree ensembles as a mixed-integer linear program. The formulation is exact but NP-hard in general and accepts only linear constraints, so it cannot express the nonlinear rate relations between bytes and duration that flow records obey. SAT and MaxSAT encodings of tree ensembles [57, 58] provide formal explanations and robustness checks, and LT-Attack [59] attacks tree ensembles through continuous leaf approximations. The MILP and LT-Attack operate in unconstrained feature space and do not enforce coupled network invariants. Robust training of tree ensembles [60, 61, 62] and adversarial retraining of neural NIDS [63] improve robustness within the same feature-space threat model. Black-box query methods such as ZOO [9] and HopSkipJump [10] estimate the decision boundary by sampling the classifier's output, and population or swarm searches mutate parameters without knowing which splits are active.

Table 1 summarizes these paradigms and the gap they leave. Methods that exploit the structure of tree ensembles or work in unconstrained feature space do not produce flows that survive packet parsing, while methods that preserve physical feasibility either treat the detector as a black box and need hundreds of queries per flow, or train a surrogate and transfer, which is cheap per flow but depends on how well the surrogate mimics the target. Flow-Field Evasion (FFE) addresses this gap by steering flows along infinitesimal packet-level generators that keep every step on the flow manifold, and by

using the split geometry of the trees to choose the direction.

2.2. Valid Packet Streams and Flow Telemetry

A packet stream $P = ((p_1, t_1), \ldots, (p_m, t_m))$ is an ordered sequence of packets with $0 \leq t_1 \leq \cdots \leq t_m$, where t_i is the transmission time at the network interface card. Only streams in the admissible set $\mathcal{P}_{\text{valid}}$ are considered. Every packet must carry valid IPv4, TCP and UDP checksums [64, 65, 66] and obey the interface MTU:

$$\forall i \in \{1, \ldots, m\}, \quad \text{Len}_{\text{IP}}(p_i) \leq \text{MTU}_{\text{IP}} = 1500 \text{ bytes} \tag{1}$$

A flow is the bidirectional set of packets sharing a 5-tuple, split into forward (client-to-server) and backward packets. A flow exporter such as CICFlowMeter [1] computes a deterministic, many-to-one mapping

$$\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d \tag{2}$$

and $x = \Phi(P)$ summarizes the transmission.

2.3. Twelve-Dimensional Feature Schema

We use a standardized 12-dimensional schema of the core metrics that state-of-the-art models analyze on CSE-CIC-IDS2018 [1]. The feature vector $x \in \mathbb{R}^{12}$ comprises: x_0, destination port; x_1, protocol (6 for TCP, 17 for UDP); x_2, number of forward packets; x_3, number of backward packets; $x_4 = \sum_{p_i \in P_{\text{fwd}}} \text{Len}(p_i)$, forward wire bytes including IP and transport headers; x_5, backward bytes; $x_6 = (t_m - t_1) \times 10^6$, flow duration in microseconds; x_7, initial forward TCP window ($0 \leq x_7 \leq 65535$, and $x_7 \equiv 0$ for UDP); x_8, flow bytes per second; x_9, flow packets per second; x_{10}, mean forward packet

Table 1: Representative evasion, verification and defense paradigms for tabular and network telemetry. Physical feasibility reports the empirical PFR of this benchmark where it was evaluated (Section 4.2).

Method	Target model	Engine	Queries	Coupled invariants	Physical feasibility
Kantchelian et al. [56]	GBDT, random forest	MILP (worst case $2^{\mathcal{O}(DT)}$)	0 (white-box)	Unsupported (linear only)	Unconstrained
Zhang et al. [59] (LT-Attack)	Tree ensembles	Continuous leaf approximation	$O(N_{\text{iter}})$	Not modeled	Unconstrained
Chen et al. (ZOO) [9]	Black-box classifiers	Zeroth-order coordinate descent	76–137 (mean)	Not modeled	14.9–35.6% (empirical)
Chen et al. (HSJ) [10]	Black-box decision	Boundary binary walk	47 (mean)	Not modeled	0.0% (empirical)
Lin et al. (IDS-GAN) [28]; Zolbayar et al. [14]	Black-box NIDS	Learned generator on a surrogate	2–5 after training	Repaired after generation	100.0% (after repair)
Ghamizi et al. (MOEVA) [12]	Constrained tabular	Multi-objective genetic search	250–434 (mean)	Heuristic penalty	48.5–67.5% (empirical)
Simonetto et al. (CAA/CAPGD) [11]	Tabular models	Constrained APGD (surrogate) with MOEVA	1–15 (CAPGD, mean)	Repair operator	100.0% (after repair)
Han et al. (Traffic Manipulator) [13]	Black-box NIDS	PSO on packet knobs	161–262 (mean)	Invariant projection	100.0% (projected)
Flow-Field Evasion (ours)	**XGBoost, LightGBM**	**Lie vector field steering**	$\leq 4M_{\max}+1$ **probes + TSBS (mean 13.0–91.4)**	**Exact closed-form**	**100.0% (schema)**

Note: T trees, D depth, $M_{\max} = 20$, $K = 4$ generators. Empirical query counts and PFR values are ranges over the three full-scale benchmarks against XGBoost (Table 3). LT-Attack is cited as white-box context and was not run; CAPGD and the NIDSGAN-style generator are our re-implementations (Section 4.1).

length; and x_{11}, forward transport header bytes, which must be at least the per-packet minimum header size times x_2. The rates and mean are fixed by the relation

$$x_8 = g_8(x_4, x_5, x_6) = \frac{10^6\,(x_4 + x_5)}{x_6}, \qquad x_9 = g_9(x_2, x_3, x_6) = \frac{10^6\,(x_2 + x_3)}{x_6},$$
$$x_{10} = g_{10}(x_4, x_2) = \frac{x_4}{\max(1, x_2)} \tag{3}$$

2.4. Feature Invariants

The indices $\mathcal{D} = \{0, \ldots, 11\}$ fall into five constraint classes derived from network semantics, and a feature may belong to more than one class.

- $\mathcal{I}_{\text{imm}} = \{0, 1, 3, 5\}$: port, protocol and backward packets and bytes are fixed by the service and the server, so $\Delta x_i = 0$.
- $\mathcal{I}^{+}_{\text{mono}} = \{2, 4, 6, 11\}$: an attacker may add packets, pad payloads or delay transmissions, but cannot remove bytes the payload needs, so $\Delta x_j \geq 0$.
- $\mathcal{I}_{\text{bound}} = \{7\}$: the window stays within $0 \leq x_7 + \Delta x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}$.
- $\mathcal{I}_{\text{dep}} = \{8, 9, 10\}$: these features are the deterministic functions g_8, g_9, g_{10} of the base metrics given above.
- $\mathcal{I}_{\text{ineq}} = \{4, 11\}$: $x_{11} \geq x_2(20\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17})$ and $0 \leq x_4 \leq 1500 x_2$.

Feature x_4 is both monotone and MTU bounded. The set $\mathcal{I}^{+}_{\text{mono}}$ gives the mutable monotone features that Algorithm 2 modifies.

2.5. Flow Manifold and Ambient Relaxation

The physical flow space is the discrete image of valid packet streams under the telemetry exporter:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \exists P \in \mathcal{P}_{\text{valid}} \text{ s.t. } \Phi(P) = x \right\} \subset \mathbb{Z}^9_{\geq 0} \times \mathbb{R}^3_{\geq 0} \tag{4}$$

where discrete coordinates $(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_{11})$ satisfy integrality (with packet counts $x_2 \in \mathbb{N}_{\geq 1}, x_3 \in \mathbb{N}_{\geq 0}$, duration in integer microseconds $1 \leq x_6 \leq 1.1 \times 10^8$ μs bounded below the 120-second exporter timeout), rates $(x_8, x_9, x_{10}) \in \mathbb{R}^3_{\geq 0}$, and packet lengths comply with interface MTU restrictions ($x_4 \leq 1500 x_2$). We focus on IPv4 ($L_{\text{IPv4}} = 20$ bytes) and standard Ethernet MTU.

To enable continuous differential geometric optimization, we define the *continuous ambient flow manifold* $\widetilde{\mathcal{M}}_{\text{flow}} \subset \mathbb{R}^{12}$ by relaxing packet counts and duration to positive reals ($x_2 \geq 1, x_6 > 0$), with transport header limits $H_{\min}(x_1) = 20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$ and $H_{\max}(x_1) = 60 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$:

$$\widetilde{\mathcal{M}}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \begin{array}{l} x_0 \in [1, 65535], \quad x_1 \in \{6, 17\}, \quad x_2 \geq 1, \quad x_3, x_5 \geq 0, \\ H_{\min}(x_1) x_2 \leq x_{11} \leq H_{\max}(x_1) x_2, \\ 0 \leq x_4 \leq 1500 x_2, \quad 0 < x_6 \leq 1.1 \times 10^8, \\ 0 \leq x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}, \quad x_{10} = x_4 / x_2, \\ x_8 = 10^6 (x_4 + x_5)/x_6, \quad x_9 = 10^6 (x_2 + x_3)/x_6 \end{array} \right\} \tag{5}$$

Continuous vector field flows are formulated on $\widetilde{\mathcal{M}}_{\text{flow}}$, while discrete execution snaps continuous trajectories back to $\mathcal{M}_{\text{flow}}$ via the closed-form projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 3.4).

2.6. Threat Model

The adversary starts from a malicious flow $x_{\text{orig}} = \Phi(P) \in \mathcal{M}_{\text{flow}}$ classified as an attack, $P(\text{Malicious} \mid x_{\text{orig}}) \geq 0.5 \iff F(x_{\text{orig}}) \geq 0$, and seeks a modified flow $x^* = \Phi(P^*) \in \mathcal{M}_{\text{flow}}$ with

$$P(\text{Malicious} \mid x^*) < 0.5 \iff F(x^*) < 0 \tag{6}$$

Tier 1 (white-box) grants access to the ensemble architecture, leaf weights and split thresholds Θ_j, which enables exact Tree-Split Boundary Snapping (TSBS) and forms the primary benchmark (Section 4.2). Tier 2 (black-box) allows only query access or zero-knowledge transfer of flows crafted on a surrogate ensemble, which Supplementary Section S5 evaluates.

The transformed trace P^* must preserve the malicious semantics, $\mathcal{S}(P^*) = \mathcal{S}(P)$, so that exploit delivery, shellcode execution, credential cracking or service degradation still succeeds on the victim. Payload padding preserves discrete payload exploits by appending non-executable, protocol-compliant bytes (SSH ignore messages, RFC 4253 [67]). Temporal dilation still delivers every packet, although it lowers the burst intensity of volumetric floods (Section 6.1); we do not test that the attack's effect on the victim is preserved.

The NIDS is an ingress biflow sensor (for example, an IPFIX collector at a border router or DMZ firewall). The adversary controls only the client endpoint, so it changes the forward features $x_2, x_4, x_6, x_7, x_{11}$ and the derived rates, while the server-generated x_3 and x_5 are immutable. Forward chaff is sent as pure ACKs with seq = SND.NXT and ack = RCV.NXT. A TCP endpoint never acknowledges a pure ACK (RFC 9293 [65], § 3.8.1), so these packets elicit no backward packets or bytes and $\Delta x_3 = \Delta x_5 = 0$.

Chaff is spaced or interleaved with forward payload and window updates so that three duplicate ACKs do not trigger fast retransmit (RFC 5681 [68], § 3.2). Window morphing edits the window field of the initial SYN consistently with window scaling (RFC 7323 [69]). For encrypted protocols such as SSH, padding is inserted at the application layer before cryptographic encapsulation (RFC 4253 [67], § 11.2).

The adversary minimizes the total operational cost

$$\min \quad \mathcal{C}(x^*, x_{\text{orig}}) = \sum_{k=1}^{4} c_k s_k \tag{7}$$

where s_k is the amount of generator k applied and c_k is its unit cost in transmission latency, bandwidth expansion and socket buffer allocation.

3. The Flow-Field Evasion (FFE) Framework

Figure 1 summarizes the framework.

3.1. Geometry of the Flow Manifold and Its Generators

The physical flow set $\mathcal{M}_{\text{flow}} = \Phi(\mathcal{P}_{\text{valid}})$ is a discrete lattice in $\mathbb{R}^{12}$. Its differential geometry is analyzed through the smooth relaxation $\widetilde{\mathcal{M}}_{\text{flow}}$ (Section 2.2), defined as the zero set of the conservation residual Ψ below, subject to $x_2 \geq 1$, $0 < x_6 \leq 1.1 \times 10^8$, $x_{11} \geq x_2(20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17})$ and $0 \leq x_4 \leq 1500 x_2$.

$$\Psi(x) = \begin{bmatrix} x_8 - \dfrac{10^6(x_4 + x_5)}{x_6} \\ x_9 - \dfrac{10^6(x_2 + x_3)}{x_6} \\ x_{10} - \dfrac{x_4}{x_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{8}$$

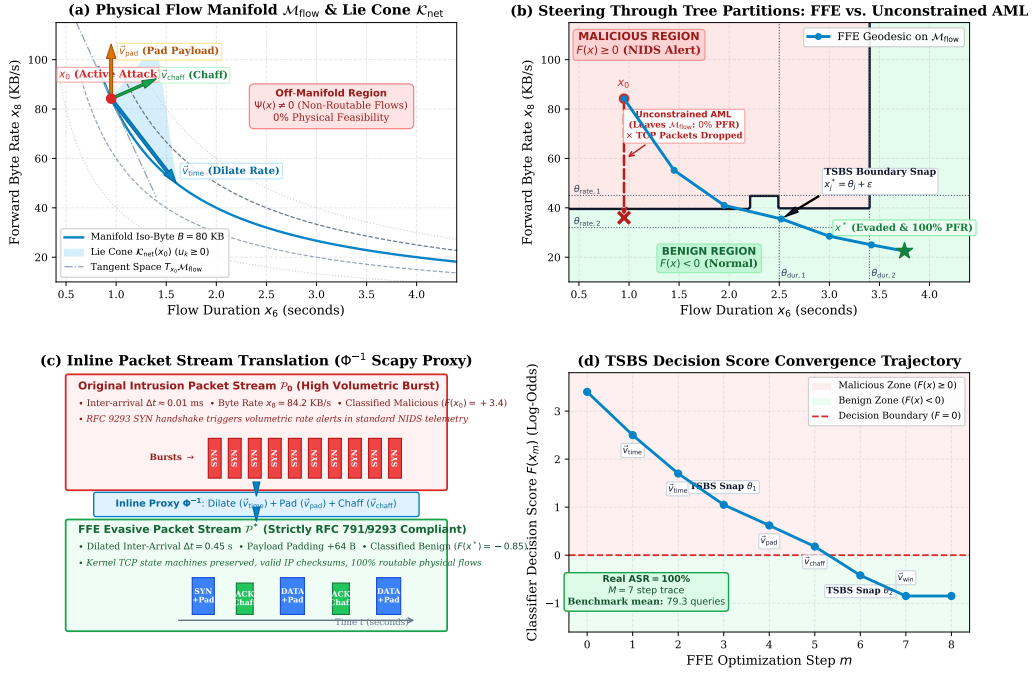


Figure 1: Architecture of the Flow-Field Evasion (FFE) framework. **(a)** The physical conservation constraints define the flow manifold $\mathcal{M}_{\text{flow}}$, and the generators span a forward Lie cone that keeps trajectories on it. **(b)** Axis-aligned tree partitions push unconstrained feature-space perturbations off the manifold, whereas FFE moves along the manifold and finishes with tree-split boundary snapping (TSBS). **(c)** Packet-level realization of the flow transformations through an inline proxy. **(d)** Decision score descent over the steps of an attack trace.

By the Regular Value Theorem, at any point where the Jacobian $J_\Psi(x) \in \mathbb{R}^{3\times 12}$ has full rank 3, the tangent space is $T_x\widetilde{\mathcal{M}}_{\text{flow}} = \ker J_\Psi(x)$. A physically realizable velocity $\vec{v} = [\dot{x}_0, \ldots, \dot{x}_{11}]^T$ therefore satisfies the linearized

conservation laws:

$$\dot{x}_8 = \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6 \tag{9}$$

$$\dot{x}_9 = \frac{10^6(\dot{x}_2 + \dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6 \tag{10}$$

$$\dot{x}_{10} = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2} \tag{11}$$

An adversary controls four elementary degrees of freedom, each generating a 1-parameter flow whose push-forward is a tangent vector field. Here $\vec{e}_k$ is the kth standard basis vector of $\mathbb{R}^{12}$ (indexed from 0):

$$\begin{aligned}
\vec{v}_{\text{time}} &= \vec{e}_6 - \frac{x_8}{x_6}\vec{e}_8 - \frac{x_9}{x_6}\vec{e}_9, \\
\vec{v}_{\text{pad}} &= \vec{e}_4 + \frac{10^6}{x_6}\vec{e}_8 + \frac{1}{x_2}\vec{e}_{10}, \\
\vec{v}_{\text{chaff}} &= \vec{e}_2 + L_0\vec{e}_4 + \frac{L_0 \cdot 10^6}{x_6}\vec{e}_8 + \frac{10^6}{x_6}\vec{e}_9 + \frac{L_0 - x_{10}}{x_2}\vec{e}_{10} + H_0\vec{e}_{11}, \\
\vec{v}_{\text{win}} &= \mathbb{I}_{x_1=6}\ \text{sat}\left(\frac{W_{\text{target}} - x_7}{\epsilon_{\text{win}}}\right)\vec{e}_7,
\end{aligned} \tag{12}$$

where temporal dilation adds delay without bytes or packets, payload padding adds forward dummy bytes, chaff injection adds forward zero-payload probes, and window morphing moves the client window toward W_{target}. Here $\text{sat}(z) = \max(-1, \min(1, z))$ and $\epsilon_{\text{win}} > 0$. The probe sizes are $L_0 = 40$, $H_0 = 20$ for empty TCP segments per RFC 9293 §3.8.4 [65], $L_0 = 52$, $H_0 = 32$ with TCP timestamps [69], and $L_0 = 28$, $H_0 = 8$ for UDP [66]. Probes are classified at the ingress perimeter before server ACKs alter backward counts; bidirectional sessions are discussed in Section 6.1. The window field is globally Lipschitz rather than C^∞ and depends only on x_1 and x_7. The implementation (Supplementary Algorithm S1) uses its $\epsilon_{\text{win}} \to 0$ limit, $\mathbb{I}_{x_1=6}\,\text{sign}(W_{\text{target}} - x_7)\,\vec{e}_7$, which moves by $|W_{\text{target}} - x_7|$ and lands exactly on W_{target} without overshoot.

3.2. Commutativity, Invariants and the Lie Cone

The Lie bracket of vector fields is $[X, Y] = J_Y X - J_X Y$, where J_X and J_Y are their Jacobians.

Proposition 1 (Commutativity and Integrability of Physical Flow Generators)**.** *Because protocol $x_1 \in \{6, 17\}$ is an immutable attribute ($\dot{x}_1 \equiv 0$), the ambient manifold $\widetilde{\mathcal{M}}_{flow}$ decomposes into two disjoint smooth invariant sub-manifolds: $\widetilde{\mathcal{M}}^{(6)}_{flow}$ (TCP flows) and $\widetilde{\mathcal{M}}^{(17)}_{flow}$ (UDP flows). On each component, the indicator $\mathbb{I}_{x_1=6}$ is constant, and L_0, H_0 are constant along any trajectory (the implementation selects $H_0 = 32$ iff the mean forward header $x_{11}/x_2 \geq 26$; injecting chaff of header size H_0 moves that mean toward H_0 and never across the threshold, so the selection is invariant under the flow), and denominators satisfy $x_2 \geq 1, x_6 > 0$, ensuring $\vec{v}_{time}, \vec{v}_{pad}, \vec{v}_{chaff}$ are C^∞ vector fields and $\vec{v}_{win}$ is globally Lipschitz (its brackets exist almost everywhere and vanish there, because x_7 is neither read nor written by the other three generators). The Lie brackets between all four infinitesimal packet transformation generators vanish identically on $\widetilde{\mathcal{M}}_{flow}$:*

$$[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0} \in \mathbb{R}^{12}, \quad \forall i, j \in \{1, 2, 3, 4\} \tag{13}$$

Consequently, the generators span an Abelian Lie algebra $\mathfrak{g}_{flow} \cong \mathbb{R}^4$ (or $\mathbb{R}^3$ for UDP).

The proof of Proposition 1, and the Frobenius integrability corollary that follows from it (Supplementary Corollary S1), are given in the Supplementary Material (Section S1).

The admissible deformation cone is the positive span of the generators:

$$\mathcal{K}_{\text{net}}(x) = \left\{ \sum_{k=1}^{4} u_k \vec{v}_k(x) \;\middle|\; u_k \geq 0 \right\} \subset T_x \widetilde{\mathcal{M}}_{\text{flow}} \tag{14}$$

Theorem 1 (Universal Invariant Preservation). *Let $x_0 \in \widetilde{\mathcal{M}}_{flow}$ be an initial network flow vector. For any measurable and locally bounded control signal $u(t) \in L^\infty_{loc}(\mathbb{R}_{\geq 0}, \mathbb{R}^4_{\geq 0})$ satisfying the admissible boundary rules ($u_2(t) \leq (1500 - L_0)u_3(t)$ whenever $x_4(t) = 1500x_2(t)$, and $u_1(t) = 0$ whenever $x_6(t) = 1.1 \times 10^8$), the trajectory $x(t)$ defined by the dynamical system:*

$$\dot{x}(t) = \sum_{k=1}^{4} u_k(t)\vec{v}_k(x(t)), \quad x(0) = x_0 \tag{15}$$

satisfies $x(t) \in \widetilde{\mathcal{M}}_{flow}$ for all $t \geq 0$, with solutions existing uniquely and globally on $[0, \infty)$. Under discrete stepping, the continuous Lie cone guides directional descent while the closed-form analytical projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 3.4) eliminates discrete $\mathcal{O}(\Delta^2)$ truncation drift and enforces packet and byte integrality ($x_2 \in \mathbb{N}_{\geq 1}, x_4, x_6 \in \mathbb{N}$) and MTU clamping $x_4 \leq 1500x_2$, ensuring that the discrete state sequence remains strictly in the physical packet manifold $\mathcal{M}_{flow}$.

Proof sketch. Differentiating Ψ along the flow gives a linear homogeneous system $\dot{\Psi} = A(t)\Psi$ with diagonal, locally integrable $A(t)$, so $\Psi \equiv \mathbf{0}$ by Carathéodory's theorem [70]. The bounds $g_i \geq 0$ are affine, so the Nagumo–Brezis condition [71, 72] holds on every face and the manifold is forward invariant. The same theorem gives existence and uniqueness of global solutions. The full proof is in the Supplementary Material (Section S1).

3.3. Operational Cost and Optimal Control

The unit costs are $c_1 = 10^{-4}$ per second of induced delay, $c_2 = 10^{-2}$ per byte of padding, $c_3 = 1.0$ per probe packet, and $c_4 = 10^{-3}$ per normalized window displacement $s_4 = |W_{\text{target}} - x_7|/65535 \in [0, 1]$. The evasion problem

is an optimal control problem:

$$\min_{u(t)\geq 0} \quad \int_0^{T^*} \left(\sum_{k=1}^{4} c_k u_k(t) \right) dt \tag{16}$$

$$\text{subject to} \quad \dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_{\text{orig}} \tag{17}$$

$$F(x(T^*)) < 0 \quad \text{(Classifier Misclassification)} \tag{18}$$

Because the generators commute (Proposition 1), the displacement depends only on $s_k = \int_0^{T^*} u_k(t)\, dt \geq 0$, and the cost $\mathcal{C}(s) = \sum_{k=1}^{4} c_k s_k$ is linear and path-independent. The tree ensemble boundary $F(x) = 0$ is piecewise constant and non-convex, so FFE greedily follows the generator with the largest directional score decrease per unit cost, $-\nabla_k / c_k$, and finishes with boundary snapping (TSBS).

3.4. Algorithmic Realization of Flow-Field Evasion

Algorithm 1 gives the master loop of Flow-Field Evasion (FFE). Starting from a flagged flow $x_0 \in \mathcal{M}_{\text{flow}}$ with $F(x_0) \geq 0$, each iteration probes the four generators, takes the admissible step with the largest score reduction per unit cost, and falls back to boundary snapping or temporal dilation when no generator lowers the score.

Supplementary Algorithm S1 estimates the directional derivatives by probing the ensemble once per generator, since ∇F vanishes almost everywhere. The probe steps are $\Delta s_1 = \max(1000,\, 0.5x_6)$ μs, $\Delta s_2 = \max(50,\, 0.25x_4)$ bytes, $\Delta s_3 = \max(1,\, \text{round}(0.5x_2))$ packets and $\Delta s_4 = |64240 - x_7|$, and the derivatives are taken per second of delay, per byte, per packet and per window unit of $\Delta s_4/65{,}535$, respectively. Every displacement is followed by

Algorithm 1 Flow-Field Evasion (FFE) Master Optimization Loop

Require: Classifier F, initial flow $x_0 \in \mathcal{M}_{\text{flow}}$, schema $\mathcal{S}$, step budget M, costs $\vec{c} = [c_1, \ldots, c_4]^T$, threshold $\tau = 0$
Ensure: Evasive flow $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < \tau$, or FAILURE

$x \leftarrow \mathcal{P}_{\mathcal{M}}(x_0)$; $f \leftarrow F(x)$; $q \leftarrow 1$
for step $= 1$ **to** M **do**
  if $f < \tau$ **then**
    return $(x, \text{SUCCESS})$
  end if
  $\mathcal{D}, q_{\text{used}} \leftarrow \text{EstimateLieDerivatives}(F, x, \mathcal{S})$; $q \leftarrow q + q_{\text{used}}$ {Supplementary Algorithm S1}
  $k^* \leftarrow$ first maximizer of $(-\nabla_k)/c_k$ over $(k, \nabla_k, \vec{v}_k, \Delta s_k, f_k) \in \mathcal{D}$ with $\nabla_k < 0$; null if none
  if $k^* \neq$ null **then**
    $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta s_{k^*}\, \vec{v}_{k^*})$; $f \leftarrow f_{k^*}$ {Cached probe, no extra query}
  else
    $x_{\text{snap}} \leftarrow \text{TSBS}(F, x, \mathcal{S})$
    if $F(x_{\text{snap}}) < f$ **then**
      $x \leftarrow x_{\text{snap}}$; $f \leftarrow F(x_{\text{snap}})$
    else
      $\Delta t \leftarrow \min(1.1 \times 10^8, \max(5 \times 10^4, 1.5\, x_6))$ {Fallback dilation; projection clamps duration at the 1.1×10^8 μs timeout}
      $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta t\, \vec{v}_{\text{time}}(x))$; $f \leftarrow F(x)$; $q \leftarrow q + 1$
    end if
  end if
end for
if $f < \tau$ **then**
  return $(x, \text{SUCCESS})$
else
  return $(x, \text{FAILURE})$
end if

the closed-form projection $\mathcal{P}_{\mathcal{M}}$ of Supplementary Algorithm S2, which restores immutable features, enforces monotonicity and integer counts, bounds header and packet lengths, and recomputes the coupled rates.

3.4.1. Tree-Split Boundary Snapping (TSBS)

Algorithm 2 implements Tree-Split Boundary Snapping (TSBS), whose candidates are generated by Supplementary Algorithm S3. Because the score

is piecewise constant in each split feature, a small displacement cannot cross a split threshold. In Tier 1 white-box settings, TSBS therefore extracts the active split thresholds of the ensemble, keeps per feature the threshold closest to the benign side of the current value, and retains the projected candidate that most decreases F.

GBDTs also split on coupled telemetry features, so TSBS inverts their constraints in closed form. To cross an active byte-rate threshold θ_8 or packet-rate threshold θ_9, it lengthens the flow duration to

$$x_6^* = \frac{10^6(x_4 + x_5)}{\theta_8} + \epsilon_6 \quad \text{or} \quad x_6^* = \frac{10^6(x_2 + x_3)}{\theta_9} + \epsilon_6, \tag{19}$$

with $\epsilon_6 = 10\,\mu$s; the candidate is discarded if x_6^* exceeds the $1.1 \times 10^8\,\mu$s cap. To cross a forward mean packet length threshold θ_{10} from above, TSBS inverts $x_{10} = (x_4 + NL_0)/(x_2 + N)$ and adds the smallest integer chaff count that satisfies the inequality,

$$N_{\text{chaff}} = \left\lfloor \frac{x_4 - \theta_{10} x_2}{\theta_{10} - L_0} \right\rfloor + 1 \qquad (L_0 < \theta_{10} < x_{10}). \tag{20}$$

For TCP flows, TSBS also snaps the window advertisement toward the benign OS baseline $x_7^* = 64{,}240$ and toward adjacent split partitions.

3.4.2. Query Accounting

Executing all $M_{\max} = 20$ iterations without early termination costs at most $Q_{\max} \leq 4M_{\max} + 1 = 81$ model queries for the baseline query and Lie-derivative probes, or ≤ 101 if stagnation fallbacks fire, plus the TSBS candidate evaluations, which are not covered by this bound; UDP flows and TCP flows whose window already equals the benign value skip window morphing and cost $q = 3$ probes per iteration. In practice, the query count

Algorithm 2 Tree-Split Boundary Snapping (TSBS)

Require: Classifier F, flow candidate $x \in \mathcal{M}_{\text{flow}}$ with $F(x) \geq \tau$ (still flagged malicious), feature schema $\mathcal{S}$, tolerance ϵ_{rel}
Ensure: Snapped flow $x^* \in \mathcal{M}_{\text{flow}}$

$x^* \leftarrow \text{copy}(x)$
active_splits $\leftarrow$ ExtractActiveSplits(F, x^*)
$\mathcal{C} \leftarrow$ GenerateCandidates$(x^*, \text{active_splits}, \epsilon_{\text{rel}})$ {Boundary-crossing pool (Supplementary Algorithm S3)}
{Evaluate projected candidates and retain best partition crossing}
$x_{\text{best}} \leftarrow x^*$; $s_{\text{best}} \leftarrow F(x^*)$
for each $x_c \in \mathcal{C}$ **do**
    $x_{\text{proj}} \leftarrow$ ProjectManifoldInvariants$(x_c, \mathcal{S})$
    if $F(x_{\text{proj}}) < s_{\text{best}}$ **then**
        $s_{\text{best}} \leftarrow F(x_{\text{proj}})$; $x_{\text{best}} \leftarrow x_{\text{proj}}$
        if $s_{\text{best}} < 0$ **then**
            break {Early exit once benign classification is achieved}
        end if
    end if
end for
return x_{best}

depends on how close the flows sit to a split boundary that a forward generator can cross: on the full benchmarks the mean is 13.0, 16.7 and 91.4 queries per flow against XGBoost on CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 (Table 3), and 10.3 queries on the balanced ablation cohort. This figure is deliberately conservative: a Tier 1 adversary could evaluate candidate TSBS split crossings offline on its local copy of the ensemble, but our evaluation counts every probe and boundary candidate evaluation as an active model query, as a strictly black-box adversary would have to. Because $K \leq 4$ and $M \leq 20$ are constants, each iteration costs $\mathcal{O}(T \cdot D)$ tree evaluations, and Table 1 compares this asymptotic cost with exact MILP and heuristic search.

4. Experiments and Results

4.1. Experimental Setup and Benchmark Corpora

Single-testbed evaluations have limited external validity [7, 24, 25, 26, 54], so we use three widely studied corpora in full: CSE-CIC-IDS2018 [1] (the nine daily "Processed Traffic Data for ML Algorithms" files, seven of which are capped at 1,048,575 rows in the distributed files), CIC-IDS2017 [1] (five daily CICFlowMeter files) and UNSW-NB15 [15] (the official training and testing partition, 257,673 records, a subset of the 2.54 million records of the complete capture). Every record is read and none is subsampled; flows are mapped onto the 12-feature schema of Section 2.2.

Cleaning is identical for all corpora. Records with missing or infinite values, non-positive duration, no forward packet, or a protocol other than TCP or UDP are dropped; records that share all 12 features and the label are collapsed to one; records that share the 12 features with conflicting labels are removed, since they have no usable ground truth. The remaining flows are split 70%/10%/20% into training, validation and test partitions, stratified by label and attack family. Deduplicating before splitting keeps any flow from appearing on both sides, a known source of inflated accuracy [49, 50, 51]. Duplicates and conflicts remove 33.5% of the valid CSE-CIC-IDS2018 records, 19.3% of CIC-IDS2017 and 44.2% of UNSW-NB15 (Table 2); the 12-feature mapping of UNSW-NB15 is lossy, so 15,702 of its records are indistinguishable from a record of the opposite class. The invariant projection of Section 2.2 is applied to every flow and changes a feature value by more than 5% in 5.8% of the CIC-IDS2017 and CSE-CIC-IDS2018 records and in none of UNSW-NB15.

Table 2: Full-scale benchmark corpora after cleaning and deduplication. Valid records pass the validity filter (UNSW-NB15: 215,742 of 257,673, the rest being non-TCP/UDP).

Dataset	Valid records	Conflicts removed	Flows kept	Attacks	Train	Test
CSE-CIC-IDS2018	6,468,390	85,740	4,300,373	543,806	3,010,261	860,075
CIC-IDS2017	2,097,825	54	1,692,949	405,355	1,185,065	338,590
UNSW-NB15	215,742	15,702	120,317	53,502	84,221	24,064

The targets are XGBoost (CUDA histogram method, trained on the whole training partition) and LightGBM, each with 60 trees of depth 6 and learning rate 0.1. Attack queries run on the CPU.

We compare FFE with seven methods. *Naive* coordinate descent in $\mathbb{R}^d$ (step $0.05\,\sigma_f$) checks no constraint. *ZOO-NIDS* [9] and *HopSkipJump-NIDS* [10] are the zeroth-order and decision-based black-box attacks. *CAA/MOEVA* [11, 12] is multi-objective evolutionary search (population 25, 20 generations). *Traffic Manipulator PSO* [13] is particle swarm search over intervals and dummy bytes (15 particles, 25 iterations); its candidates are projected onto the manifold with our closed-form invariants, so it faces the same constraints as FFE. *CAPGD* [11] is our re-implementation of constrained adaptive PGD with momentum and step halving: the attacker trains its own MLP surrogate (two hidden layers of 128 units) on the training flows, takes sign-gradient steps in a log-standardized space (L_∞ budget 1.0, no step on immutable features), repairs every candidate onto the manifold, and queries the target once per iteration (at most 20), stopping at the first evasive candidate. The *NIDSGAN-style generator* [14] is a generator network trained against the same surrogate to emit a bounded perturbation, repaired with the same operator, with at most five noise draws queried against the target; it follows the published idea but is not the authors' code. *FFE* uses step budget $M = 20$.

We did not run the white-box tree attacks of Zhang et al. [59] and Cascioli et al. [40], which need the tree structure and do not model flow constraints.

ASR is the fraction of attacks with $P(\text{Malicious} \mid x^*) < 0.5$. PFR is the fraction of candidate flows that pass the protocol, non-negativity and exact coupled-rate checks, that is, $x^* \in \mathcal{M}_{\text{flow}}$. Real ASR counts an attack only when it both evades and is feasible, Real ASR $= \frac{1}{N}\sum_i \mathbb{I}[F(x_i^*) < 0 \wedge x_i^* \in \mathcal{M}_{\text{flow}}]$. We report Wilson 95% intervals and exact McNemar tests on paired per-flow outcomes. The cost column is the mean operational cost $\sum_j c_j|x_j^* - x_j|$ with the per-feature unit costs of the schema (payload bytes and delay are cheap, forward packets cost one unit, immutable features 1,000). FFE steers by exactly this quantity and the baselines (except MOEVA) do not optimize it, so the column shows the operational price of the evasion each method finds, not an optimized head-to-head comparison.

Cohorts are drawn once with a fixed seed from the malicious test flows that the target detects. Every method except CAA/MOEVA and PSO attacks the same 1,000 flows; those two need 250–430 queries per flow and attack the first 200 of them, and the McNemar comparisons against them use these 200. The ablation, transferability and defense studies use a balanced cohort of up to 50 detected flows per family of CSE-CIC-IDS2018 (300 flows). All results use one training seed, so the intervals capture sampling variation of the cohort but not retraining variance (Section 6.1).

4.2. Master Comparative Benchmark

Table 3 compares FFE with seven baselines on the three full-scale corpora. Three observations hold throughout. First, evading in feature space is not evading with a valid flow. HopSkipJump-NIDS reaches 100.0% ASR on

Table 3: Evasion benchmark against CUDA XGBoost on the complete CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures. All methods except CAA/MOEVA and PSO attack the same 1,000 detected malicious test flows; CAA/MOEVA and PSO attack the first 200 of them. ASR, PFR and Real ASR in %; Cost is the operational cost of Section 4.1.

Method	ASR	PFR	Real ASR [95% CI]	Queries	Cost	ms/flow
CSE-CIC-IDS2018 (XGBoost: accuracy 98.04%, test attacks 108,762, detected 92,167)						
Unconstrained feature space	38.5	38.1	38.1 [35.1, 41.2]	75.2	3,064.9	84.7
ZOO-NIDS [9]	30.3	14.9	14.9 [12.8, 17.2]	115.6	3,797.5	128.3
HopSkipJump-NIDS [10]	100.0	0.0	0.0 [0.0, 0.4]	47.0	997,194.7	52.6
CAA/MOEVA [11, 12]	66.0	48.5	21.0 [15.9, 27.2]	249.8	84.394	292.6
Traffic Manipulator PSO [13]	60.0	100.0	60.0 [53.1, 66.5]	161.3	2,502.3	196.2
CAPGD (surrogate) [11]	99.5	100.0	99.5 [98.8, 99.8]	1.1	24,367.0	4.8
NIDSGAN-style [14]	70.5	100.0	70.5 [67.6, 73.2]	2.2	38,408.5	6.6
FFE (ours)	100.0	100.0	**100.0 [99.6, 100.0]**	13.0	0.127	16.9
CIC-IDS2017 (XGBoost: accuracy 99.81%, test attacks 81,071, detected 80,481)						
Unconstrained feature space	68.9	33.0	17.8 [15.6, 20.3]	36.5	37,130.9	49.8
ZOO-NIDS [9]	65.0	26.3	9.0 [7.4, 10.9]	76.4	35,141.0	83.8
HopSkipJump-NIDS [10]	100.0	0.0	0.0 [0.0, 0.4]	47.0	1,819,529.4	61.7
CAA/MOEVA [11, 12]	51.5	67.5	31.5 [25.5, 38.2]	276.5	218.2	388.3
Traffic Manipulator PSO [13]	32.0	100.0	32.0 [25.9, 38.8]	261.8	2,629.8	358.3
CAPGD (surrogate) [11]	55.2	100.0	55.2 [52.1, 58.3]	9.7	18,357.8	55.9
NIDSGAN-style [14]	55.5	100.0	55.5 [52.4, 58.6]	2.8	18,158.7	10.1
FFE (ours)	95.9	100.0	**95.9 [94.5, 97.0]**	16.7	0.315	26.0
UNSW-NB15 (XGBoost: accuracy 88.80%, test attacks 10,701, detected 9,359)						
Unconstrained feature space	16.8	50.0	9.2 [7.6, 11.2]	44.9	388.2	94.8
ZOO-NIDS [9]	15.1	35.6	0.0 [0.0, 0.4]	136.9	497.4	280.3
HopSkipJump-NIDS [10]	4.6	0.0	0.0 [0.0, 0.4]	47.0	18,935,252.8	91.3
CAA/MOEVA [11, 12]	15.5	58.0	14.5 [10.3, 20.0]	433.9	30.771	960.8
Traffic Manipulator PSO [13]	43.5	100.0	43.5 [36.8, 50.4]	226.8	2,331.9	475.2
CAPGD (surrogate) [11]	26.8	100.0	26.8 [24.1, 29.6]	15.0	5,056.2	135.3
NIDSGAN-style [14]	13.6	100.0	13.6 [11.6, 15.9]	4.5	2,487.3	18.5
FFE (ours)	61.5	100.0	**61.5 [58.4, 64.5]**	91.4	0.915	213.6

CSE-CIC-IDS2018 and CIC-IDS2017, yet none of its candidates is physically feasible, so its Real ASR is 0.0% on every corpus. The naive attack falls from 68.9% to 17.8% ASR on CIC-IDS2017 once feasibility is enforced (16.8% to 9.2% on UNSW-NB15), and ZOO-NIDS from 65.0% to 9.0% (15.1% to 0.0%). CAA/MOEVA penalizes violations but does not repair them and ends with a PFR of 48.5%–67.5%.

Second, FFE has the highest Real ASR on all three corpora: 100.0%,

95.9% and 61.5%. The margin depends on the corpus. On CIC-IDS2017 and UNSW-NB15 it is large and significant (95.9% against 55.5% for the NIDSGAN-style generator and 55.2% for CAPGD on CIC-IDS2017; 61.5% against 43.5% for PSO on UNSW-NB15, with 39 flows evaded only by FFE and 2 only by PSO among 200, exact McNemar $p < 10^{-9}$). On CSE-CIC-IDS2018 the gap closes: CAPGD reaches 99.5% against 100.0%, the five flows it misses are the only discordant pairs, and the difference is not significant (McNemar $p = 0.0625$). We therefore do not claim that FFE outperforms a surrogate-based constrained attack on that corpus. Its XGBoost model splits on destination port at the root of 55 of 60 trees, yet the decision is carried by many mutable features below the root, and almost every detected flow can be moved across the boundary with a few probes by either method.

Third, the methods trade queries, training effort and cost differently. CAPGD and the NIDSGAN-style generator are cheap at attack time (1.1–15.0 and 2.2–4.5 queries per flow) but need a trained surrogate, and their success depends on how well it resembles the target: CAPGD drops from 99.5% on CSE-CIC-IDS2018 to 26.8% on UNSW-NB15. FFE needs no surrogate but needs the split thresholds (Tier-1 knowledge) and spends 13.0, 16.7 and 91.4 queries per flow. Against the surrogate-free constrained methods (PSO and CAA/MOEVA) it needs 2.5–19 times fewer queries and 2.2–17 times less time per flow. The cost column shows the largest gap: FFE's evasions cost 0.127, 0.315 and 0.915 operational units on average, the baselines' between roughly 30 and 2×10^7, partly because FFE steers by this cost and the others do not.

Table 4: FFE against XGBoost and LightGBM on the full corpora: clean test performance of the target and FFE success on the 1,000-flow cohort (Wilson 95% intervals).

Dataset	Model	Acc. (%)	F1 (%)	AUC (%)	FFE Real ASR (%)	PFR (%)	Queries
CSE-CIC-IDS2018	XGBoost	98.04	91.62	98.07	100.0 [99.6, 100.0]	100.0	13.0
	LightGBM	98.03	91.58	98.08	100.0 [99.6, 100.0]	100.0	18.0
CIC-IDS2017	XGBoost	99.81	99.60	100.00	95.9 [94.5, 97.0]	100.0	16.7
	LightGBM	99.82	99.63	100.00	34.6 [31.7, 37.6]	100.0	116.5
UNSW-NB15	XGBoost	88.80	87.41	96.70	61.5 [58.4, 64.5]	100.0	91.4
	LightGBM	89.22	87.89	96.81	55.2 [52.1, 58.3]	100.0	99.4

4.3. Per-family and cross-dataset validation

Table 4 shows that clean accuracy no longer sits uniformly near 99% once duplicates are removed and the whole capture is used: 98.04% on CSE-CIC-IDS2018 (F1 91.6%, since 12.6% of the flows are attacks and the rare families are hard), 99.81% on CIC-IDS2017 and only 88.80% on UNSW-NB15, where the 12-feature mapping is lossy. FFE keeps a feasibility rate of 100.0% against both architectures on every corpus, but its success depends on the architecture on CIC-IDS2017, where XGBoost is evaded in 95.9% of the cohort and LightGBM in 34.6% (116.5 queries). Among the XGBoost trees of CIC-IDS2017, 26 of 60 place their root split on the protocol field, 25 on the initial forward window (a mutable feature, consistent with the high success on XGBoost) and 9 on backward payload bytes; we did not analyze the LightGBM trees and claim no mechanism for the lower LightGBM figure. On UNSW-NB15 the architectures behave alike (61.5% and 55.2%).

Per-family results are in Supplementary Table S1: every CSE-CIC-IDS2018 family is evaded in 100.0% of the flows, and on UNSW-NB15 exploits, more than half of the cohort, are evaded in only 51.0% of 520 flows and set the aggregate figure.

4.4. Ablations and transferability

The full ablation and transferability tables are in the Supplementary Material (Sections S4 and S5). We summarize them for the CUDA XGBoost target of CSE-CIC-IDS2018 and a balanced cohort of $N = 300$ detected malicious flows.

ASR rises with the step budget from 52.7% at $M = 1$ (5.0 queries) through 96.3% at $M = 5$ to 100.0% at $M = 20$ (10.3 queries), with 100.0% physical feasibility throughout. No single generator suffices: alone they evade 30.0% (padding), 47.7% (chaff), 53.0% (window morphing) and 57.0% (temporal dilation, at 33.5 queries and 39.8 s of added duration), pairs reach 49.0%–82.7%, and only the complete cone reaches 100.0%. The cost regime and the TSBS snapping offset leave ASR at 100.0%, so on this saturated corpus they are uninformative about success.

In many operational settings the attacker cannot query the deployed model. Flows crafted against XGBoost evade LightGBM in 69.0% of the cases, Random Forest in 81.3% and CatBoost in 37.7%; across the other source models, 54%–83% of transferred flows evade every target except CatBoost, which is the hardest target for every source (34.0%–37.7%) even though attacks against CatBoost itself succeed in 99.3% of the cases.

5. Defensive Countermeasures

Standard boosting picks split features by loss reduction alone, so features that separate the classes in benchmark corpora, such as duration and the rates, can sit near the root although an attacker can shift them by delaying or padding packets. We evaluate malleability-penalized splitting (MPS), which

discourages splits on malleable features, and Lie-guided adversarial training (LGAT), which trains on flows generated along the Lie generators.

5.1. Malleability-penalized tree splitting

Following robust split formulations [60, 62], the split score for feature j is regularized by its physical malleability index $M_{\text{mal}}(j)$:

$$M_{\text{mal}}(j) = \mathbb{E}_{x \sim \mathcal{D}_{\text{train}}} \left[\max_{k \in \{1,\ldots,4\}} \frac{\delta s_k |\vec{v}_k(x)_j|}{\sigma_j} \right] \tag{21}$$

The robust split selection objective is

$$\text{Gain}_{\text{robust}}(j, \theta) = \Delta\mathcal{L}(j, \theta) - \alpha_{\text{mal}} \cdot \overline{\Delta\mathcal{L}} \cdot M_{\text{mal}}(j) \tag{22}$$

where σ_j is the feature standard deviation, δs_k is the perturbation scale ($\delta s_1 = 10^4\,\mu\text{s}, \delta s_2 = 100\,\text{B}, \delta s_3 = 1\,\text{pkt}, \delta s_4 = 10^3\,\text{B}$), $\Delta\mathcal{L}(j, \theta)$ is the impurity gain, $\overline{\Delta\mathcal{L}}$ is the baseline mean gain, and $\alpha_{\text{mal}} \geq 0$ balances clean loss against malleability. The experiments evaluate the $\alpha_{\text{mal}} \to \infty$ limit by masking duration (x_6) and rates (x_8, x_9) to zero in training, test, and attack inputs.

5.2. Lie-guided adversarial training

Standard adversarial training assumes an L_p ball around each input [73], which in network security generates physically invalid telemetry that degrades clean accuracy [63, 7]. LGAT instead restricts the inner maximization to the physical flow manifold:

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \max_{u \in \mathcal{U}_{\text{budget}}} \mathcal{L}\left(F_\Theta\left(\mathcal{A}_{\text{cone}}\left(x_i, u\right)\right), y_i\right) \tag{23}$$

where $\mathcal{A}_{\text{cone}}(x, u)$ is the flow reached along Lie generator trajectories under control budget $\mathcal{U}_{\text{budget}}$. In our implementation (Supplementary Algorithm S4), FFE adversarial examples ($M = 5$) are generated against the

undefended model for a random sample of 5,000 malicious training flows (about 1.3% of the 381,000 malicious flows of the training partition, capped to keep the generation cost bounded) and appended to the training set.

5.3. Evaluation

We compared an undefended XGBoost model with three hardened variants on the full CSE-CIC-IDS2018 capture, each with 60 trees of depth 6 and learning rate 0.1, trained on the whole training partition (3.0 million flows). All models are evaluated on the balanced cohort of $N = 300$ malicious flows that the undefended model detects, using FFE at step budget $M = 5$ (Table 5). Robust recall is $100 - \text{ASR}$.

Table 5: Defensive hardening against Flow-Field Evasion on the full CSE-CIC-IDS2018 capture (XGBoost; cohort $N = 300$; attack evaluation budget $M = 5$; all values in %).

Model	**Acc. (%)**	**AUC (%)**	**Robust recall (%)**	**FFE ASR (%**
Baseline XGBoost (Undefended)	98.04	98.07	3.7	96.3
MPS-Hardened XGBoost	98.04	98.01	33.0	67.0
LGAT-Hardened XGBoost	98.04	98.06	67.0	33.0
Dual-Hardened (MPS + LGAT)	98.04	98.00	62.0	38.0

All three defenses leave clean detection unchanged (accuracy 98.04% for every model, ROC-AUC 98.00%–98.07%), so none of the robustness comes at a visible cost in clean performance on this corpus. The undefended model is evaded in 96.3% of the cohort at $M = 5$. Masking the duration and rate features (MPS) lowers this to 67.0%, Lie-guided adversarial training (LGAT) to 33.0%, and the combination to 38.0%. LGAT alone is therefore the strongest of the three, and combining it with MPS does not help: the 5-point difference between LGAT (33.0%) and MPS+LGAT (38.0%) is within

the sampling error of a 300-flow cohort (Wilson 95% intervals of roughly ±5 points) and we make no claim that one is better than the other.

Interpretation.. MPS removes the three features an attacker moves most cheaply (duration and the two rates) and evasion falls by 29 points, but the ensemble still splits on forward packet counts, payload bytes, header length and window size, which FFE moves with small chaff, padding and window changes. LGAT shows the model flows reached along the same generators and brings evasion down by 63 points even though only 1.3% of the malicious training flows were augmented. Residual evasion of 33% against an attacker who uses exactly the generators the model was trained on shows that small-budget augmentation is not a certificate of robustness. We evaluated only the attack that generated the augmentation, so the figures do not measure robustness to a different attack such as the CAPGD baseline; the defenses were also evaluated on CSE-CIC-IDS2018 only, where FFE is saturated against the undefended model.

6. Discussion and Conclusions

6.1. Operational Feasibility, Query Efficiency, and Limitations

Feature-space attacks are fast, but their output is not guaranteed to be a physically realizable flow (Section 2). FFE pays for feasibility with a different structure. Its generators act on the flow manifold, so trajectories in the continuous Lie cone satisfy the conservation and rate-coupling invariants exactly, and a projection step restores them on the discrete flow schema. All evasive flows that FFE produced passed the feasibility check (100.0% PFR on every corpus and architecture). This figure measures conformance to the

12-feature schema under discrete projection, not an independent test on live packets.

The comparison with the baselines is not a single ranking. Against the constraint-respecting query-based methods (PSO and CAA/MOEVA), FFE is better on every corpus (for example 61.5% against 43.5% for PSO on UNSW-NB15) with 2.5–19 times fewer queries, because TSBS snaps flows onto adjacent split boundaries instead of searching by stochastic dispersion. Against surrogate-based attacks it depends on the corpus. CAPGD reaches 99.5% on CSE-CIC-IDS2018 with 1.1 queries per flow, statistically indistinguishable from FFE's 100.0% at 13.0 queries, but falls to 55.2% and 26.8% on CIC-IDS2017 and UNSW-NB15, where FFE reaches 95.9% and 61.5%. An attacker with a good surrogate can therefore do very well on some corpora, and FFE's advantage is largest where the surrogate imitates the target poorly. Its cost advantage (0.13–0.92 operational units against 30 to 2×10^7) is partly by construction, since FFE steers by the cost model that defines the column.

The method has clear limits. TSBS requires Tier-1 white-box access to split thresholds. Under Tier-2 access the attack depends on probes and transfer, and flows crafted against one architecture evade other architectures in 34%–83% of the cases on CSE-CIC-IDS2018 (Supplementary Section S5), with CatBoost the hardest target. Traffic Manipulator operates on captured PCAP files, whereas our packet realization is a Scapy/NFQueue proxy that has not been replayed at line rate. Control is restricted to forward traffic, backward server features are immutable, deep packet inspection is out of scope, and rate dilution lowers the intensity of volumetric floods.

Evasion efficacy depends on the corpus and the architecture. XGBoost on CSE-CIC-IDS2018 places the root split of 55 of its 60 trees on destination port, an immutable feature, and is nevertheless evaded in every attacked flow, so immutability of the root feature does not by itself protect a model. On CIC-IDS2017 XGBoost is evaded in 95.9% of the cohort and LightGBM in 34.6%; we offer no mechanism for this difference. UNSW-NB15 is the hardest corpus for all attacks, although its lossy 12-feature mapping also makes the detector weaker (88.8% accuracy).

Threats to validity are as follows. Each experiment uses one training seed, so the intervals reflect cohort sampling and not retraining variance. Cohorts contain only flows that the target detects. CAA/MOEVA and PSO ran on 200 flows, the ablations and defenses on 300, the others on 1,000. CAPGD and the NIDSGAN-style generator are our re-implementations with one small surrogate and one hyperparameter setting, so a better-tuned surrogate attack could do better. "Full scale" refers to the distributed processed files (CSE-CIC-IDS2018 files are capped at 1,048,575 rows; the UNSW-NB15 files are the official partition). Deduplication removes 19%–44% of the valid records, so a detector trained on raw, duplicated traffic would see a different distribution, and the 12-feature schema makes accuracies incomparable with all-feature studies. The defenses were evaluated only against the attack that produced their augmentation and only on CSE-CIC-IDS2018.

Each generator has a protocol-compliant packet realization: sender pacing for temporal dilation, padding in protocol fields or ignored application frames (`SSH_MSG_IGNORE`) [67], payload-free pure ACK or keep-alive chaff [65, 69] or zero-length UDP datagrams [66], and handshake window morphing, with

sequence numbers and checksums updated via proxy [64, 65, 66].

6.2. Concluding Remarks and Future Directions

This article has measured, at full scale, how vulnerable tree-ensemble NIDS with high benchmark accuracy are to evasion that respects flow physics. Flow-Field Evasion models four packet-level actions as infinitesimal generators on the flow manifold; trajectories in the continuous Lie cone preserve the conservation invariants and a projection enforces them on the discrete schema. On the complete, deduplicated CSE-CIC-IDS2018, CIC-IDS2017 and UNSW-NB15 captures, FFE evades XGBoost in 100.0%, 95.9% and 61.5% of detected attacks with 100.0% physical feasibility and 13.0–91.4 queries per flow. It is significantly more successful and cheaper in queries than the constraint-respecting black-box baselines, significantly more successful than surrogate-based CAPGD on two corpora, and statistically tied with it on the third, where CAPGD needs fewer queries. Evasion is not a property of the attack alone: LightGBM on CIC-IDS2017 resists it far better than XGBoost, and CatBoost resists transfer from the other ensembles. Lie-guided adversarial training reduces FFE success from 96.3% to 33.0% without affecting clean accuracy, whereas masking malleable features helps less (67.0%) and adds nothing on top of adversarial training. Future work includes multi-seed evaluation, defenses that withstand attacks other than the one used for augmentation, the remaining public NIDS corpora, line-rate replay on a hardware testbed, and certified robustness bounds for adversarially trained ensembles.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no external funding.

Data availability

The code, dataset extraction scripts, and reproducible experimental pipelines, together with all benchmark result archives for FFE, the baseline methods (Naive, ZOO-NIDS, HopSkipJump-NIDS, CAA/MOEVA, and Traffic Manipulator PSO), ablations, transferability, and defenses, are publicly available at https://github.com/vinhqdang/adversarial_ids. All flow data originate from public benchmark archives (CSE-CIC-IDS2018, UNSW-NB15, and CIC-IDS2017).

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 3146–3154, neurIPS proceedings carry no DOI.
URL https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, CatBoost: Unbiased boosting with categorical features, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018, pp. 6638–6648. doi:10.48550/arXiv.1706.09516.

[5] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[6] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.

[7] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, ACM Digital Threats: Research and Practice 3 (3) (2022) 1–19. doi:10.1145/3469659.

[8] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[9] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), 2017, pp. 15–26. doi:10.1145/3128572.3140448.

[10] J. Chen, M. I. Jordan, M. J. Wainwright, HopSkipJumpAttack: A query-efficient decision-based attack, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1277–1294. doi:10.1109/SP40000.2020.00045.

[11] T. Simonetto, S. Ghamizi, M. Cordy, Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 27817–27849. doi:10.52202/079017-0873.

[12] S. Ghamizi, M. Cordy, M. Papadakis, Y. L. Traon, Search-based adversarial testing and improvement of constrained credit scoring systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1089–1100. doi:10.1145/3368089.3409739.

[13] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, X. Yin, Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors, IEEE Journal on Selected Areas in Communications 39 (8) (2021) 2632–2647. doi:10.1109/JSAC.2021.3087242.

[14] B.-E. Zolbayar, R. Sheatsley, P. McDaniel, M. J. Weisman, S. Zhu, S. Zhu, S. Krishnamurthy, Generating practical adversarial network traffic flows using NIDSGAN (2022). arXiv:2203.06694, doi:10.48550/arXiv.2203.06694.

[15] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[16] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, A. Hotho, Flow-based benchmark data sets for intrusion detection, in: Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS), 2017, pp. 361–369, original CIDDS-001 dataset paper; no DOI assigned.
URL https://www.hs-coburg.de/forschung/forschungsprojekte-oeffentlich/

informationstechnologien/cidds-coburg-intrusion-detection-data-sets.html

[17] Q.-V. Dang, Using machine learning for intrusion detection systems, Computing and Informatics 41 (1) (2022) 12–33. doi:10.31577/cai_2022_1_12.

[18] Q.-V. Dang, Multi-layer intrusion detection on the USB-IDS-1 dataset, in: Hybrid Intelligent Systems (HIS 2022), Vol. 647 of Lecture Notes in Networks and Systems, Springer, Cham, 2023, pp. 1114–1121. doi:10.1007/978-3-031-27409-1_102.

[19] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014. doi:10.48550/arXiv.1312.6199.

[20] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[21] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, A. Swami, The limitations of deep learning in adversarial settings, in: Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 372–387. doi:10.1109/EuroSP.2016.36.

[22] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

[23] M. Verkerken, L. D'hooge, B. Volckaert, F. D. Turck, G. Apruzzese, "What is the problem space?" Defining host-space adversarial perturbations against network intrusion detection systems, in: Proceedings of the 21st ACM ASIA Conference on Computer and Communications Security (ASIA CCS), 2026, pp. 1043–1059. doi:10.1145/3779208.3807482.

[24] G. Apruzzese, H. S. Anderson, L. Cavallaro, "Real attackers don't compute gradients": Bridging the gap between adversarial ML research and practice in cybersecurity, in: Proceedings of the 1st IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), 2023, pp. 339–364. doi:10.1109/SaTML54575.2023.00031.

[25] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, IEEE Communications Surveys & Tutorials 25 (1) (2023) 538–566. doi:10.1109/COMST.2022.3233793.

[26] J. Vitorino, I. Praça, E. Maia, SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection, Computers & Security 134 (2023) 103433. doi:10.1016/j.cose.2023.103433.

[27] T. Simonetto, S. Ghamizi, M. Cordy, TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 78394–78430. doi:10.52202/079017-2492.

[28] Z. Lin, Y. Shi, Z. Xue, IDSGAN: Generative adversarial networks for attack generation against intrusion detection, in: Advances in Knowledge Discovery and Data Mining (PAKDD), Lecture Notes in Computer Science, 2022, pp. 79–91. doi:10.1007/978-3-031-05981-0_7.

[29] M. A. Merzouk, F. Cuppens, N. Boulahia-Cuppens, R. Yaich, Investigating the

practicality of adversarial evasion attacks on network intrusion detection, Annals of Telecommunications 77 (2022) 763–775. doi:10.1007/s12243-022-00910-1.

[30] R. Sheatsley, N. Papernot, M. Weisman, G. Verma, P. McDaniel, Adversarial examples for network intrusion detection systems, Journal of Computer Security 30 (2022) 727–752. doi:10.3233/JCS-210094.

[31] I. Debicha, B. Cochez, T. Kenaza, T. Debatty, J.-M. Dricot, W. Mees, Adv-Bot: Realistic adversarial botnet attacks against network intrusion detection systems, Computers & Security 129 (2023) 103176. doi:10.1016/j.cose.2023.103176.

[32] I. Debicha, T. Kenaza, I. Charfi, S. Mosbah, M. Sehaki, J.-M. Dricot, Targeted adversarial traffic generation: Black-box approach to evade intrusion detection systems in IoT networks, International Journal of Machine Learning and Cybernetics 17 (2026) 58. doi:10.1007/s13042-025-02873-w.

[33] H. Zhang, D. Han, Y. Liu, Z. Wang, J. Sun, S. Zhuang, J. Liu, J. Dong, Explainable and transferable adversarial attack for ML-based network intrusion detectors, IEEE Transactions on Dependable and Secure Computing 22 (2025) 5090–5107. doi:10.1109/TDSC.2025.3560486.

[34] M. elShehaby, A. Matrawy, A novel perturb-ability score to mitigate evasion adversarial attacks on flow-based ML-NIDS, Journal of Information Security and Applications 99 (2026) 104409. doi:10.1016/j.jisa.2026.104409.

[35] K. He, D. D. Kim, J. Sun, J. D. Yoo, Y. H. Lee, H. K. Kim, Liuer mihou: A practical framework for generating and evaluating grey-box adversarial attacks against NIDS (2022). arXiv:2204.06113, doi:10.48550/arXiv.2204.06113.

[36] K. Kireev, B. Kulynych, C. Troncoso, Adversarial robustness for tabular data through cost and utility awareness, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2023. doi:10.14722/ndss.2023.24924.

[37] Z. He, C. Ouyang, L. Wen, C. Liu, C. Moreira, TabAttackBench: A benchmark for adversarial attacks on tabular data, Expert Systems with Applications 301 (2026) 130491. doi:10.1016/j.eswa.2025.130491.

[38] S. Dyrmishi, M. Djilani, T. Simonetto, S. Ghamizi, M. Cordy, Insights on adversarial attacks for tabular machine learning via a systematic literature review (2025). arXiv:2506.15506, doi:10.48550/arXiv.2506.15506.

[39] J. Vitorino, M. Silva, E. Maia, I. Praça, An adversarial robustness benchmark for enterprise network intrusion detection, in: Foundations and Practice of Security (FPS 2023), Lecture Notes in Computer Science, 2024, pp. 3–17. doi:10.1007/978-3-031-57537-2_1.

[40] L. Cascioli, L. Devos, O. Kuželka, J. Davis, Faster repeated evasion attacks in tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 24762–24794. doi:10.52202/079017-0780.

[41] A. Pashamokhtari, G. Batista, H. H. Gharakheili, AdIoTack: Quantifying and refining resilience of decision tree ensemble inference models against adversarial volumetric attacks on IoT networks, Computers & Security 120 (2022) 102801. doi:10.1016/j.cose.2022.102801.

[42] L. Devos, L. Perini, W. Meert, J. Davis, Detecting evasion attacks in deployed tree ensembles, in: Machine Learning and Knowledge Discovery in Databases: Research

Track (ECML PKDD 2023), Lecture Notes in Computer Science, 2023, pp. 120–136. doi:10.1007/978-3-031-43424-2_8.

[43] D. Vos, S. Verwer, Efficient training of robust decision trees against adversarial examples, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, 2021, pp. 10586–10595. doi:10.48550/arXiv.2012.10438.

[44] D. Vos, S. Verwer, Robust optimal classification trees against adversarial examples, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 8520–8528. doi:10.1609/aaai.v36i8.20829.

[45] S. Calzavara, C. Lucchese, F. Marcuzzi, S. Orlando, Feature partitioning for robust tree ensembles and their certification in adversarial scenarios, EURASIP Journal on Information Security 2021 (2021) 12. doi:10.1186/s13635-021-00127-0.

[46] S. Calzavara, L. Cazzaro, G. E. Pibiri, N. Prezza, Verifiable learning for robust tree ensembles, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023, pp. 1850–1864. doi:10.1145/3576915.3623100.

[47] S. Calzavara, L. Cazzaro, C. Lucchese, G. E. Pibiri, Verifiable boosted tree ensembles, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 2057–2075. doi:10.1109/SP61157.2025.00022.

[48] B. Lindqvist, On the adversarial robustness of decision trees and a symmetry defense, IEEE Access 13 (2025) 16120–16132. doi:10.1109/ACCESS.2025.3530695.

[49] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the CICIDS2017 case study, in: 2021 IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12. doi:10.1109/SPW53761.2021.00009.

[50] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, E. Totel, Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes, in: Risks and Security of Internet and Systems (CRiSIS 2022), Lecture Notes in Computer Science, 2023, pp. 18–33. doi:10.1007/978-3-031-31108-6_2.

[51] L. Liu, G. Engelen, T. Lynar, D. Essam, W. Joosen, Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018, in: 2022 IEEE Conference on Communications and Network Security (CNS), 2022, pp. 254–262. doi:10.1109/CNS56114.2022.9947235.

[52] R. Flood, D. Aspinall, Measuring the complexity of benchmark NIDS datasets via spectral analysis, in: 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2024, pp. 335–341. doi:10.1109/EuroSPW61312.2024.00043.

[53] S. Layeghy, M. Gallagher, M. Portmann, Benchmarking the benchmark – comparing synthetic and real-world network IDS datasets, Journal of Information Security and Applications 80 (2024) 103689. doi:10.1016/j.jisa.2023.103689.

[54] M. Cantone, C. Marrocco, A. Bria, Machine learning in network intrusion detection: A cross-dataset generalization study, IEEE Access 12 (2024) 144489–144508. doi:10.1109/ACCESS.2024.3472907.

[55] X. F. Guo, X. Zheng, A. M. Peñuela, L. Cavallaro, S. Maffeis, F. Pierazzi, KnowML: Improving generalization of ML-NIDS with attack knowledge graphs, in: 2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P), 2026, pp. 929–948. doi:10.1109/EuroSP68448.2026.00064.

[56] A. Kantchelian, J. D. Tygar, A. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2387–2396. doi:10.48550/arXiv.1509.07892.

[57] A. Ignatiev, Y. Izza, P. J. Stuckey, J. Marques-Silva, Using MaxSAT for efficient explanations of tree ensembles, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 3776–3785. doi:10.1609/aaai.v36i4.20292.

[58] Y. Izza, A. Ignatiev, J. Marques-Silva, On tackling explanation redundancy in decision trees, Journal of Artificial Intelligence Research 75 (2022) 261–321. doi:10.1613/jair.1.13575.

[59] C. Zhang, H. Zhang, C.-J. Hsieh, An efficient adversarial attack for tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 16165–16176. doi:10.48550/arXiv.2010.11598.

[60] H. Chen, H. Zhang, S. Si, Y. Li, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 1122–1131. doi:10.48550/arXiv.1902.10660.

[61] M. Andriushchenko, M. Hein, Provably robust boosted decision stumps and trees against adversarial attacks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 12997–13008. doi:10.48550/arXiv.1906.03526.

[62] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: training evasion-aware decision trees, Data Mining and Knowledge Discovery 34 (5) (2020) 1390–1420. doi:10.1007/s10618-020-00694-9.

[63] M. J. Hashemi, E. Keller, Enhancing robustness against adversarial examples in network intrusion detection systems, in: Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), 2020, pp. 37–43. doi:10.1109/NFV-SDN50289.2020.9289869.

[64] J. Postel, Internet protocol, RFC 791, RFC Editor (1981). doi:10.17487/RFC0791.

[65] W. Eddy, Transmission control protocol (TCP) specification, RFC 9293, RFC Editor (2022). doi:10.17487/RFC9293.

[66] J. Postel, User datagram protocol, RFC 768, RFC Editor (1980). doi:10.17487/RFC0768.

[67] T. Ylonen, C. Lonvick, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (2006). doi:10.17487/RFC4253.

[68] M. Allman, V. Paxson, E. Blanton, TCP congestion control, RFC 5681, RFC Editor (2009). doi:10.17487/RFC5681.

[69] D. Borman, B. Braden, V. Jacobson, R. Scheffenegger, TCP extensions for high performance, RFC 7323, RFC Editor (2014). doi:10.17487/RFC7323.

[70] C. Carathéodory, Vorlesungen über reelle Funktionen, B. G. Teubner, Leipzig, 1918.

[71] M. Nagumo, Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen, Proceedings of the Physico-Mathematical Society of Japan. 3rd Series 24 (1942) 551–559. doi:10.11429/ppmsj1919.24.0_551.

[72] H. Brézis, On a characterization of flow-invariant sets, Communications on Pure and Applied Mathematics 23 (2) (1970) 261–263. doi:10.1002/cpa.3160230211.

[73] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018. doi:10.48550/arXiv.1706.06083.